

successful, the function returns zero and the scheduler will in the future take the affinity information into account.

Analysis setup

htop is used to analyse the process execution flow. The parameters, like the CPU number, Utime (the user CPU time, which is the amount of time the process has taken to execute on the CPU in user mode), Stime (the system CPU time, which is the amount of time the kernel has spent executing system calls on behalf of the process, measured in clock ticks) and the time for each application process, are observed.

Please note that the *htop* gets process information from a file called *stat* under each process' *procfs* entry. For processes running a real-time scheduling policy (see *sched_setscheduler(2)*), this is the negated scheduling priority, minus one; that is, a number in the range of -2 to -100, corresponding to real-time priorities 1 to 99.

Hence, the priority of 5 is displayed as -6; that of 3 as -4 and 2 as -3 in *htop* results. Therefore, all figures have process priorities as -6, -4 and -3.

Along with this, *dmesg* is used to check the kernel's response to application processes.

Results and observations

Let us perform a small experiment on a character device driver to check semaphore and rt-mutex lock behaviour at the kernel level. The basic idea is that a device can be accessed by multiple application programs, where two or more devices need to update a shared memory or buffer. In this case, how a pre-emptible kernel behaves is what we learned during this experiment.

The experiment is performed on two platforms – on a single core processor and multi-core processor system.

Three simple application programs are written to check the kernel lock behaviour. Two of the applications try to access the device driver code (which is our critical section) and the third process does not access the device driver.

These three applications are written with different static priorities assigned to them. Let's name these as low (priority 2), middle (priority 3) and high (priority 5).

Uniprocessor systems

Without lock setup: A low priority process is executed first. Then the high priority process comes in, which also wants to write in the same buffer.

Observation: The high priority process pre-empt the low priority process. So, when two processes are executed, the buffer is overwritten by the higher priority process. Thus, when a lower priority process tries to read the buffer content, it gets wrong data.

Semaphore

The setup: A low priority process is executed first. Then a high priority process comes in which also wants to write to the same buffer. A middle priority process comes in which does not want to access the device driver. Middle is a simple application program.

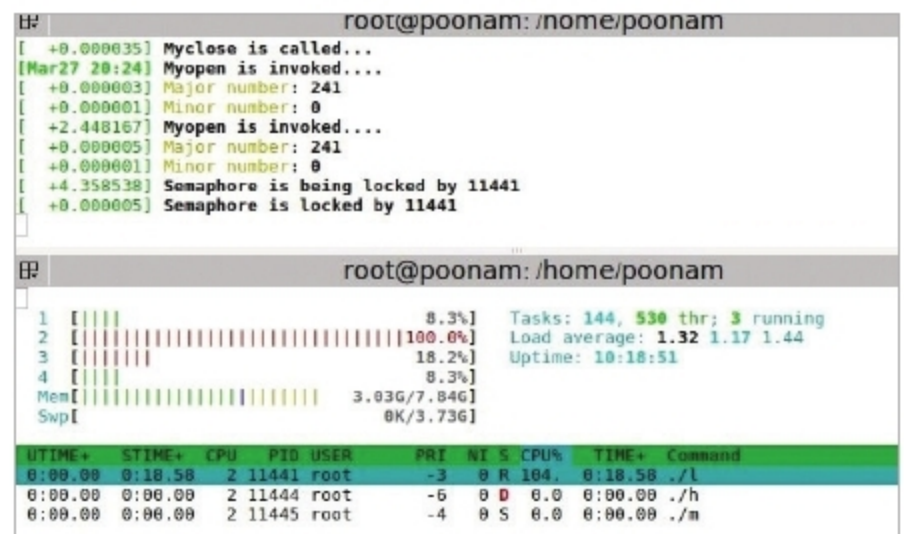


Figure 5: Low executes, high waits for low to release CS

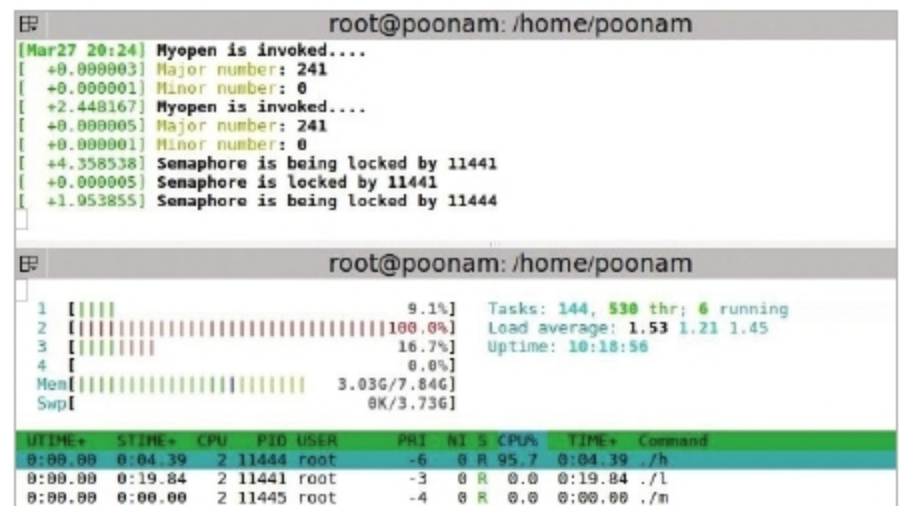


Figure 6: High executes after low completes execution in CS

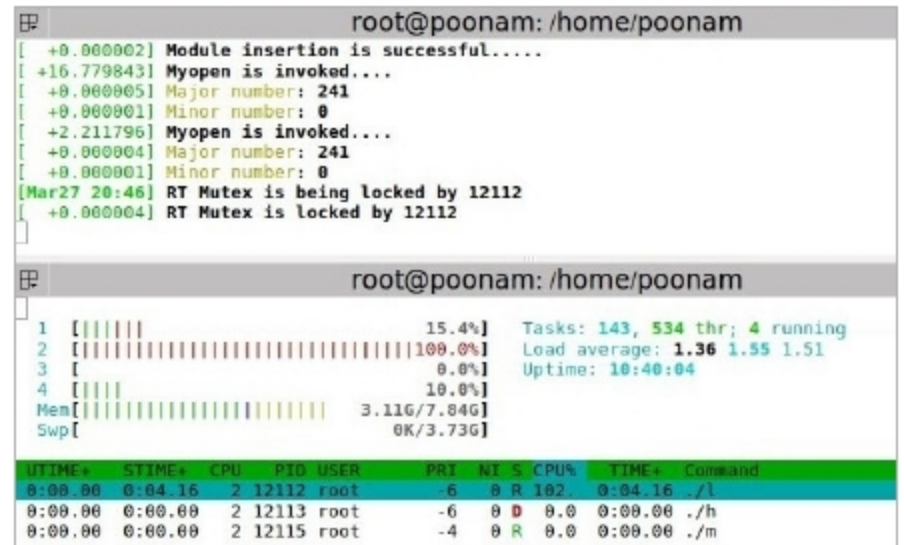


Figure 7: Low process priority gets inherited

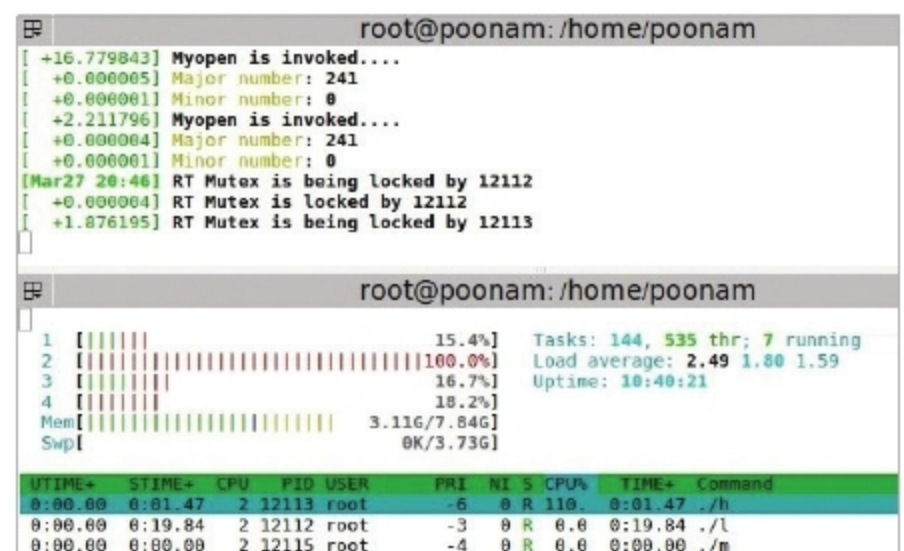


Figure 8: Inherited priority gets reset after execution